

Time & Space Complexity

A Practical Reference Guide with Worked Examples

This guide gives you a clear, one-line definition of every Big-O complexity class you'll encounter in day-to-day programming and interviews, a runnable code example for each, and a step-by-step method for calculating time and space complexity yourself. Examples use a single running scenario — **ShopEasy**, a retail company operating across Indian cities — so the numbers stay concrete throughout.

1. How to Calculate Time Complexity — Step by Step

Follow these four steps on any piece of code. Do this in order — don't try to guess the final answer before counting the operations.

Step 1 — Count the loops

A single loop that runs once per element is $O(n)$. A loop that runs a fixed number of times regardless of input (e.g. `for i in range(5)`) is $O(1)$, not $O(n)$.

Step 2 — Nested loops multiply

A loop inside another loop, both depending on the same input size n , multiplies: $n \times n = n^2$. Three nested loops over n give n^3 , and so on.

Step 3 — Consecutive (back-to-back) loops add

Two separate loops that each run over the data, one after another, ADD: $O(n) + O(n) = O(2n)$. They do not multiply. This is the single most common mix-up.

Step 4 — Drop constants and lower-order terms

Once you have a total like $O(2n)$ or $O(n^2 + n)$, simplify by dropping constants ($2n \rightarrow n$) and keeping only the fastest-growing term ($n^2 + n \rightarrow n^2$). Big-O only cares about behaviour as n gets very large.

Worked example

```
for x in arr:           # loop 1: runs n times -> O(n)
    print(x)

for y in arr:           # loop 2: runs n times -> O(n)
    print(y)

# Total work = n + n = 2n
# Drop the constant -> O(n) (NOT O(n^2) -- these loops are consecutive, not nested)
```

Practical takeaway: if two loops are written one after another at the same indentation level, they add. If one is written inside the other, they multiply. Indentation is your clue.

2. How to Calculate Space Complexity — Step by Step

Space complexity asks: how much *extra* memory does the algorithm need, beyond the input itself, as input size grows? Use these steps.

Step 1 — Identify variables that don't grow with n

Simple counters, running totals, and loop variables (i , j , x) are $O(1)$ — they take the same single slot of memory no matter how large the input is.

Step 2 — Identify data structures that DO grow with n

A new list, set, or dictionary built from the input (e.g. filtering into a new list, or building a lookup map) grows in proportion to $n \rightarrow O(n)$ space.

Step 3 — Account for the call stack in recursion

Every recursive call adds a new frame to the call stack, and that frame stays in memory until the call returns. A recursive function that calls itself n times before returning uses $O(n)$ space for the stack — even if it never creates a single list or dictionary.

Step 4 — Decide whether to count the input itself

By convention, we usually report *extra* (auxiliary) space — memory used beyond the input. Be explicit about which one you mean when you state an answer.

Worked example — the recursion trap

```
def countdown(n):
    if n == 0:
        return
    countdown(n - 1)

# Time:  $O(n)$  -> the function is called  $n$  times
# Space:  $O(n)$  -> NOT  $O(1)$ . Each call sits on the call stack until it
#           returns, so  $n$  calls means  $n$  stack frames exist at once.
```

Watch out: students very often get the time complexity right here and the space complexity wrong. “No lists or dictionaries were created” does not mean $O(1)$ space when recursion is involved.

3. The Complexity Classes: Definitions & Examples

Ordered from fastest to slowest. For each class: a plain-English definition, a ShopEasy example, the code, and exactly how the time/space figure is calculated.

O(1) — Constant Time

Definition: The number of operations stays exactly the same no matter how large the input gets.

ShopEasy scenario: Looking up an employee's city using their Employee ID in a dictionary.

```
employees = {"E101": "Bengaluru", "E102": "Pune", "E103": "Hyderabad"}
print(employees["E102"]) # Pune
```

How to calculate the TIME complexity

There is exactly one operation — a hash lookup — regardless of whether the dictionary has 3 entries or 3 crore entries. No loop, no dependency on $n \rightarrow O(1)$.

How to calculate the SPACE complexity

Only the returned value is used; no new structure is created that scales with $n \rightarrow O(1)$ extra space.

Practical takeaway: if you ever write a loop that scans a list to find a match by key, ask whether a dictionary lookup could replace it — that's an $O(n) \rightarrow O(1)$ upgrade.

O(log n) — Logarithmic Time

Definition: Each step eliminates roughly half of the remaining data, so the number of steps grows very slowly as input grows.

ShopEasy scenario: Finding a target pincode in a sorted list of 1 million pincodes using binary search.

```
def binary_search(sorted_pins, target):
    low, high = 0, len(sorted_pins) - 1
    while low <= high:
        mid = (low + high) // 2
        if sorted_pins[mid] == target:
            return mid
        elif sorted_pins[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1
```

How to calculate the TIME complexity

Each loop iteration cuts the search range in half. Starting from n items, after k halvings only $n / 2^k$ items remain; the loop ends when that reaches 1, i.e. $k = \log_2(n)$. For $n = 1,000,000$, that's only about 20 steps $\rightarrow O(\log n)$.

How to calculate the SPACE complexity

Only a few variables (low, high, mid) are used, regardless of $n \rightarrow O(1)$.

Practical takeaway: binary search only works on sorted data. If your data isn't sorted, sorting it first costs $O(n \log n)$ — only worth it if you'll search many times.

O(n) — Linear Time

Definition: The number of operations grows in direct, one-to-one proportion with the input size.

ShopEasy scenario: Adding up the amount of every order to get total revenue.

```
total = 0
for amount in orders:
    total += amount
print(total)
```

How to calculate the TIME complexity

The loop body runs exactly once per element. 10 orders → 10 iterations; 10,000 orders → 10,000 iterations → **$O(n)$** .

How to calculate the SPACE complexity

Only one extra variable (`total`) is used, and it doesn't grow as the order list grows → **$O(1)$** extra space.

$O(n \log n)$ — Log-Linear Time

Definition: Slightly worse than linear — typically seen in efficient sorting algorithms that repeatedly split data in half and then merge it back together.

ShopEasy scenario: Sorting orders by amount to find the top spenders.

```
orders = [("ORD-1", 1200), ("ORD-2", 8500), ("ORD-3", 340)]
top_orders = sorted(orders, key=lambda x: x[1], reverse=True)
```

How to calculate the TIME complexity

Python's `sorted()` (Timsort) repeatedly splits the list in half — that's $\log_2(n)$ levels — and at each level merges all n elements back together, which costs $O(n)$ per level. $\log(n)$ levels $\times$ $O(n)$ per level = **$O(n \log n)$** .

How to calculate the SPACE complexity

Merging typically needs a temporary array proportional to the input → **$O(n)$** auxiliary space for Python's Timsort in the worst case.

Watch out: a common mistake is assuming `sorted()` is $O(n)$ because “it only looks at the list once.” It effectively passes over the data $\log(n)$ times, not once.

$O(n^2)$ — Quadratic Time

Definition: The number of operations grows with the square of the input size — usually from comparing every element to every other element.

ShopEasy scenario: Finding duplicate customer names by comparing every customer to every other customer.

```
duplicates = []
for i in range(len(customers)):
    for j in range(len(customers)):
        if i != j and customers[i] == customers[j]:
            duplicates.append(customers[i])
```

How to calculate the TIME complexity

The outer loop runs n times; for every single one of those iterations, the inner loop also runs n times. Total operations = $n \times n = n^2 \rightarrow$ **$O(n^2)$** . If the data doubles, the work roughly quadruples.

How to calculate the SPACE complexity

The duplicates list could, in the worst case, hold close to n items $\rightarrow O(n)$ extra space (the nested loop itself adds no extra space, only extra time).

Practical takeaway: the same problem can usually be solved in $O(n)$ time using a set to track items already seen — trading a bit of memory for a large speed gain.

$O(2^n)$ — Exponential Time

Definition: Every additional unit of input roughly doubles the total work. This is what naive, non-memoized recursion looks like when a function calls itself more than once per call.

ShopEasy scenario: Computing Fibonacci numbers recursively without caching results.

```
def fib(n):
    if n <= 1:
        return n
    return fib(n - 1) + fib(n - 2)
```

How to calculate the TIME complexity

Each call to `fib(n)` makes two further calls, `fib(n-1)` and `fib(n-2)`, and this branching repeats all the way down. The number of calls roughly doubles with every increase in $n \rightarrow O(2^n)$. At $n = 30$ this is already over a million calls.

How to calculate the SPACE complexity

The maximum depth of the call stack at any one time is n (the longest chain of calls before hitting the base case) $\rightarrow O(n)$ space, even though the total number of calls is exponential.

Practical takeaway: this is almost always fixable with caching (memoization) — storing each `fib(k)` result the first time it's computed reduces this to $O(n)$ time.

4. Big-O Simplification Rules

Once you've counted the raw operations, simplify using these rules — Big-O only describes behaviour as n becomes very large, so small terms and constants stop mattering.

Raw expression	Simplifies to	Why
$O(n + 10)$	$O(n)$	The constant +10 becomes insignificant next to n at scale.
$O(2n)$	$O(n)$	Constant multipliers don't change the growth shape.
$O(n^2 + n)$	$O(n^2)$	n^2 dominates n once n is large — keep only the fastest-growing term.

Two mix-ups to avoid

1. Consecutive loops add, nested loops multiply.

```
for x in arr: ...      # O(n)
for y in arr: ...      # O(n)
# ADD:  $O(n) + O(n) = O(2n) \rightarrow O(n)$  (loops are separate, not nested)
```

2. Dictionary/hash lookups are $O(1)$ on average, not always. In the rare worst case (many hash collisions), a lookup can degrade to $O(n)$. Saying “ $O(1)$ average case” is the technically correct phrasing.

5. Quick Reference Cheat Sheet

Class	Time	Space	ShopEasy example
Constant	$O(1)$	$O(1)$	Get employee by ID
Logarithmic	$O(\log n)$	$O(1)$	Binary search a sorted pincode list
Linear	$O(n)$	$O(1)$	Sum all order amounts
Linear (new structure)	$O(n)$	$O(n)$	Filter even-numbered order IDs into a new list
Log-linear	$O(n \log n)$	$O(n)$	Sort orders by amount
Quadratic	$O(n^2)$	$O(1)$ – $O(n)$	Naive duplicate customer detection
Exponential	$O(2^n)$	$O(n)$	Fibonacci without memoization

Growth at a glance (operations needed for a given n)

n	$O(1)$	$O(\log n)$	$O(n)$	$O(n \log n)$	$O(n^2)$
10	1	3	10	33	100
100	1	7	100	664	10,000
1,000	1	10	1,000	9,966	1,000,000
10,000	1	13	10,000	132,877	100,000,000

At $n = 10,000$, an $O(n^2)$ algorithm needs 100 million operations while $O(n)$ needs only 10,000 — a 10,000× difference from the same input.

6. Three Traps Worth Memorising

1. Recursion has a hidden space cost

Time and space are not always the same for recursive code. A function that calls itself n times before returning is $O(n)$ time *and* $O(n)$ space, because every call sits on the call stack until the recursion unwinds.

2. Nested loops over the same data structure are the biggest performance red flag

Whenever you see one loop inside another, both iterating over data whose size depends on n , check whether a dictionary or set could remove the inner loop and bring the whole thing down to $O(n)$.

3. “Fast” and “memory-efficient” are different questions

An algorithm can be fast (good time complexity) while quietly consuming huge amounts of memory (poor space complexity), or vice-versa. Always state both when analysing code — e.g. caching makes repeated calls fast but memory usage grows with every new input cached.

Companion resource: see the accompanying Jupyter notebook for the same concepts with live, runnable, timed code demonstrations.